****

PRACTICAL NO : 03

TITLE : Implement Min, Max, Sum and Average operations using Parallel Reduction.

NAME : Pranjali Anandrao Rasal

CLASS : BE-B COMPUTER

ROLL NO :33

DATE :26/4 /2026 **BATCH : B**

****

*******INPUT*********

Main Program

```
import time
from multiprocessing import Pool, cpu_count

# Worker Function
def partial_compute(sub_arr):
    return (
        min(sub_arr),
        max(sub_arr),
        sum(sub_arr),
        len(sub_arr)
    )

# Parallel Reduction
def parallel_reduction(arr):
    n = len(arr)
    workers = cpu_count()

    chunk_size = max(1, n // workers)
    chunks = [arr[i:i + chunk_size] for i in range(0, n, chunk_size)]

    with Pool() as pool:
        results = pool.map(partial_compute, chunks)

    # Combine results
    global_min = float('inf')
    global_max = float('-inf')
    total_sum = 0
    total_count = 0

    for mn, mx, sm, cnt in results:
        global_min = min(global_min, mn)
```

```
        global_max = max(global_max, mx)
        total_sum += sm
        total_count += cnt

    avg = total_sum / total_count

    return global_min, global_max, total_sum, avg

# - Sequential Version
def sequential_reduction(arr):
    return min(arr), max(arr), sum(arr), sum(arr) / len(arr)

def main():
    # User Input
    n = int(input("Enter number of elements: "))
    arr = list(map(int, input("Enter elements: ").split()))

    #  Sequential
    start = time.time()
    s_min, s_max, s_sum, s_avg = sequential_reduction(arr)
    end = time.time()

    print("\nSequential Results:")
    print("Min:", s_min, "Max:", s_max, "Sum:", s_sum, "Avg:", s_avg)
    print("Time:", end - start, "seconds")

    # Parallel
    start = time.time()
    p_min, p_max, p_sum, p_avg = parallel_reduction(arr)
    end = time.time()

    print("\nParallel Results:")
    print("Min:", p_min, "Max:", p_max, "Sum:", p_sum, "Avg:", p_avg)
    print("Time:", end - start, "seconds")

if __name___== "__main__":
    main()
```

*******OUTPUT*********

```
PS E:\python> & "C:/Program Files/Python313/python.exe" e:/python/udmey/Oop/Min_max.py
Enter number of elements: 5
Enter elements: 4 9 5 3 7

Sequential Results:
Min: 3 Max: 9 Sum: 28 Avg: 5.6
Time: 2.1457672119140625e-05 seconds

Parallel Results:
Min: 3 Max: 9 Sum: 28 Avg: 5.6
Time: 0.19573521614074707 seconds
PS E:\python>
```

MET's Institute Of Engineering, NASHIK. *Prepared By:*